CS 61B Spring 2021

Disjoint Sets and Asymptotics

Discussion 6: February 22, 2021

1 Disjoint Sets, a.k.a. Union Find

In lecture, we discussed the Disjoint Sets ADT. Some authors call this the Union Find ADT. Today, we will use union find terminology so that you have seen both.

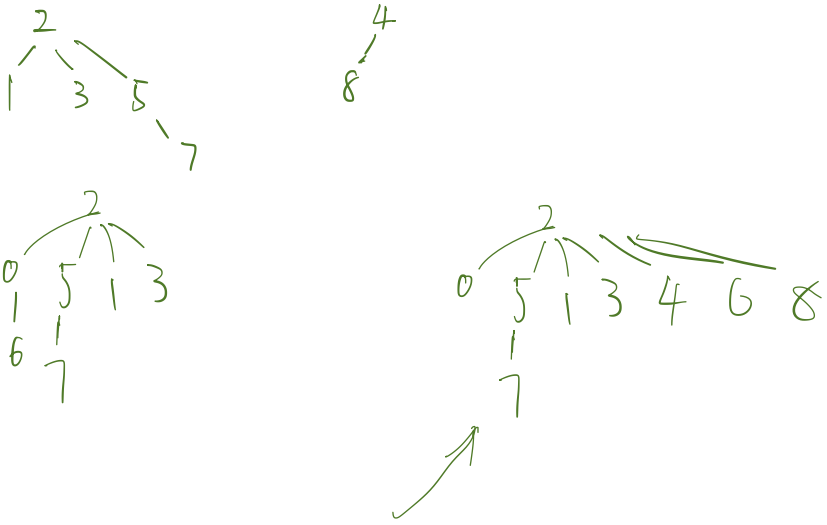
(a) What are the last two improvements (out of four) that we made to our naive implementation of the Union Find ADT during lecture 14 (Monday's lecture)?

1. Improvement 1: connect the tree by weight
2. Improvement 2: compress the path

(b) Assume we have nine items, represented by integers 0 through 8. All items are initially unconnected to each other. Draw the union find tree, draw its array representation after the series of `connect()` and `find()` operations, and write down the result of `find()` operations using **WeightedQuickUnion** without path compression. Break ties by choosing the smaller integer to be the root.

Note: `find(x)` returns the root of the tree for item `x`.

```
connect(2, 3);
connect(1, 2);
connect(5, 7);
connect(8, 4);
connect(7, 2);
find(3);  -> 2
connect(0, 6);
connect(6, 4);
connect(6, 3);
find(8);  -> 2
find(6);  -> 2
```



(c) *Extra:* Repeat the above part, using **WeightedQuickUnion with Path Compression**.

(d) What is the runtime for "connect" and "isConnected" operations using our Quick Find, Quick Union, and Weighted Quick Union ADTs? Can you explain why the Weighted Quick union has better runtimes for these operations than the regular Quick Union?

	Quick Find	Quick Union	WQU
connect	O(N)	O(N)	O(logN)
IsConnected	O(1)	O(N)	O(logN)

The height of tree is limited to logN, so WQU has better runtimes.

2 Asymptotics

(a) Order the following big-O runtimes from smallest to largest.

$$O(\log n), O(1), O(n^n), O(n^3), O(n \log n), O(n), O(n!), O(2^n), O(n^2 \log n)$$

$O(1)$ $O(\log n)$ $O(n)$ $O(n \log n)$ $O(n^2 \log n)$ $O(n^3)$ $O(2^n)$ $O(n!)$ $O(n^n)$

(b) Are the statements in the right column true or false? If false, correct the asymptotic notation ($\Omega(\cdot)$, $\Theta(\cdot)$, $O(\cdot)$). Be sure to give the tightest bound. $\Omega(\cdot)$ is the opposite of $O(\cdot)$, i.e. $f(n) \in \Omega(g(n)) \iff g(n) \in O(f(n))$.

We can see Ω as $\geq$, O as $\leq$, Θ as $=$

Hint: Make sure to simplify the runtimes first.

i) $f(n) = 20501$	$g(n) = 1$	$f(n) \in O(g(n))$	Θ more accurate
ii) $f(n) = n^2 + n$	$g(n) = 0.000001n^3$	$f(n) \in \Omega(g(n))$	O
iii) $f(n) = 2^{2n} + 1000$	$g(n) = 4^n + n^{100}$	$f(n) \in O(g(n))$	Θ more accurate
iv) $f(n) = \log(n^{100})$	$g(n) = n \log n$	$f(n) \in \Theta(g(n))$	O ✓
v) $f(n) = n \log n + 3^n + n$	$g(n) = n^2 + n + \log n$	$f(n) \in \Omega(g(n))$	✓
vi) $f(n) = n \log n + n^2$	$g(n) = \log n + n^2$	$f(n) \in \Theta(g(n))$	✓
vii) $f(n) = n \log n$	$g(n) = (\log n)^2$	$f(n) \in O(g(n))$	Ω more accurate

$100 \log n$

4^n vs n^{100}, this dominate

i) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

ii) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

iii) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

iv) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

v) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

vi) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

vii) True, this an accurate tightest bound. False, an accurate tightest bound would be _____

(c) Give the worst case and best case runtime in terms of M and N. Assume `ping` is in $\Theta(1)$ and returns an **`int`**.

```
1  for (int i = N; i > 0; i--) {
2      for (int j = 0; j <= M; j++) {
3          if (ping(i, j) > 64) break;
4      }
5  }
```

best $\Theta(N)$

worst $\Theta(MN)$

(d) Below we have a function that returns true if every int has a duplicate in the array, and false if there is any unique int in the array. Assume `sort(array)` is in $\Theta(N \log N)$ and returns `array` sorted.

```
1  public static boolean noUniques(int[] array) {
2      array = sort(array);
3      int N = array.length;
4      for (int i = 0; i < N; i += 1) {
5          boolean hasDuplicate = false;
6          for (int j = 0; j < N; j += 1) {
7              if (i != j && array[i] == array[j]) {
8                  hasDuplicate = true;
9              }
10         }
11         if (!hasDuplicate) return false;
12     }
13     return true;
14 }
```

1. Give the worst case and best case runtime where $N =$ `array.length`.

   worst case: $\Theta(N^2)$

   best case: $\Theta(N)$

2. Try to come up with a way to implement `noUniques()` that runs in $\Theta(NlogN)$ time. Can we get any faster?

   Sort it first. Sorting needs $\Theta(N \log N)$;

   And traverse the array. $\Theta(N)$;

   $\Rightarrow \Theta(N \log N)$

   Using hashMap() we can get $\Theta(N)$